

$$\frac{x}{a} + \frac{y}{b} = 1$$

$$ax^2 + bx + c = 0$$



$$f(x)$$

$$y - y_1 = m(x - x_1)$$

시선추적 기반 수학 풀이 습관 진단 및 메타인지 강화 시스템 캡스톤 4차발표

FHTC 20231345 신윤서 / 20231347 이윤지

Facial & Habitual Tracking Core

2026.06.19

— Contents

01	한 학기 프로젝트 진행 상황 정리
02	3차 한계점 리마인드 및 구현 목표
03	문제 유형 분류 제거 결정 + LLM 모델 선정 + 출력 확인
04	프롬프트 엔지니어링 4단계 + 인프라 트러블 슈팅
05	구현 데모
06	시선추적 모듈 연동
07	AOI 시스템 연동
08	최종 통합 리포트 예시
09	한 학기 성취 요약 + 한계점 및 향후 계획

한 학기 프로젝트 진행 상황 정리

- **1차 (3월) - 문제 정의**

- 5대 오답 패턴 정의, 시선추적 기반 진단 컨셉 설계

- **2차 (5월) - 핵심 아키텍처 구현**

- useGazeTracker / focusLog / patternDetector 분리하는 아키텍처 구현
- AOI 4개 영역 설계, 5대 패턴 1차 탐지 로직 구현

- **3차 (5월) - 데이터 연동 + LLM 1차 도입**

- AI Hub 데이터 변환 파이프라인 구축
- LLM 관계어 추출 + 패턴 판정 연동

- **4차 (6월) - LLM 판별 두뇌 고도화 + 실제 시선 추적 연동**

- 문제 유형 분류 제거 → LLM 직접 판단 구조로 전환 → QwQ-32B 기반 LLM Ideal Path 생성
- 마우스 대체 방식 → Chrome 확장 프로그램 실시간 시선 추적으로 대체

3차 한계점 리마인드 및 구현 목표

[3차에서 발견된 한계]

① 문제 유형 분류 오류 | 키워드 매칭

→ 복합 유형 문제 오분류 (문장제인데 "나눗셈" 키워드로 연산유형 오분류)

② LLM 풀백의 정보 부족

→ LLM에 충분한 컨텍스트 전달 안 됨/ 애매한 케이스일수록 LLM이 도움이 안 되는 상황

③ 시선 추적 미구현

→ 마우스 호버로 시선을 대체 (실제 시선 데이터 X)

[구현 목표]

→ 유형 분류 단계를 제거 & 문제 텍스트를 LLM이 직접 전체 맥락으로 판단하게 전환

→ LLM 파이프라인 자체를 검증하고 고도화

→ 기존 AOI 시스템(2~3차)에 LLM 두뇌를 연결하는 것이 이번 목표

문제 유형 분류 제거

[처음 설계 의도]

- 문제 유형을 먼저 분류해두면 LLM이 그 유형에 맞춰 핵심 구간을 더 수월하게 판단할 거라 예상
- 유형 분류 단계를 먼저 두고 LLM에 전달하는 구조로 시작

[실제 적용 후 결과]

- 키워드 매칭 분류가 LLM 호출 이전 단계에 존재 → **이미 잘못 분류된 채로 LLM 전달되는 문제**
- 미리 정한 틀이 LLM이 문맥 전체를 보고 판단하는 것을 제약

[변경된 구조]

- 유형 분류 단계 삭제 → 문제 텍스트 전체를 LLM에 그대로 전달
- 핵심 구간이 어디인지 LLM이 문맥을 보고 직접 판단

 **구조적으로 분류 오류라는 원천 자체를 제거**

— 모델 선정 1차 : Qwen2.5-Math-7B

[모델 소개]

- 알리바바 QWEN 시리즈 중 수학 문제 풀이 특화 7B 모델
- 수학 벤치마크에서 높은 성능
- 파라미터 수 적어 경량 서버에서도 구동 가능



[선택 이유]

- QWQ-32B는 GPU 메모리 20GB+ 필요
 - 서버 자원 한계로 1차 후보 제외
- 수학 도메인 특화 모델이 지시 이해에도 강할 거라 생각

[GPU 제약 해결]

- 4-BIT QUANTIZATION(BITANDBYTES)적용
 - FLOAT 16 기준 14GB → 4-BIT ~4GB로 축소, 단일 GPU 로드 성공

[결과 : 실패]

→ 문제와 무관한 텍스트 무한 생성(할루시네이션)

[실패 원인]

- 수식을 “푸는” 능력은 있으나 “핵심만 추출하라“는 INSTRUCTION FOLLOWING에 취약
- 수학 도메인 특화 ≠ 지시 이해 능력



모델 선정 2차 : QwQ-32B 전환

[모델 소개]

- 알리바바 **QWEN** 시리즈 기반 **32B 추론(REASONING)** 특화 모델
- <THINK> 태그로 내부 추론을 하는 모델
- CHAIN-OF-THOUGHT 방식 → **단계적 사고 후 최종 답변 생성**
- 복잡한 수학 추론 및 INSTRUCTION FOLLOWING 능력 우수



QwQ-32B

[Qwen7B와의 핵심 차이]

- **QWEN2.5-MATH-7B**
: 수식 계산 특화, 지시 이해 약함
- **QWQ-32B**
: 추론 과정 자체가 강점
"중요한 부분만 뽑아라"는 메타인지적 지시 정확히 수행

[GPU 제약 해결]

- 로컬 8GPU 서버
→ GPU 6~7번 모두 타 프로세스 상시 점유
- CYBERSECURITY-2G서버에 OLLAMA로 구동 중인 QWQ-32B를 HTTP API로 호출하는 방식으로 전환
- 학습이 아닌 추론만 수행 → 서버 부담 최소화

모델 선정 2차 : QwQ-32B 전환

[호출 구조]

맥 브라우저 → 8GPU 프록시 → CYBERSECURITY-2G OLLAMA → QWQ-32B

[결과 : 성공]

```
[문제] 나머지가 다른 하나는 어느 것인가요?  
[핵심 구간]  
[핵심 구간]  
1. 나머지가 다른 하나  
  
[풀이 순서]  
1. 각 항목을 같은 수로 나눈다  
2. 나머지를 비교한다  
3. 다른 나머지를 가진 것을 찾는다  
-----  
[문제] 다음 계산을 하세요. \[ 54 \times 7 \]  
[핵심 구간]  
[핵심 구간]  
1. 54  
2. ×  
3. 7  
  
[풀이 순서]  
1. 50×7 계산  
2. 4×7 계산  
3. 결과 합치기
```

→ **환각 없이 문제 표현을 그대로
활용한 정확한 핵심 구간 추출**

Qwq-32B 출력 확인

```
[문제] $3 \times 5$의 값은 무엇인가요?  
[핵심구간]  
[핵심 구간]  
1.  $3 \times 5$   
2. 값은 무엇인가요  
  
[풀이 순서]  
1. 3과 5 확인  
2. 곱셈 기호 확인  
3.  $3 \times 5$  계산  
4. 결과 도출  
-----  
[문제] 다음 계산을 하세요.  $[ 60 \times 6 ]$   
[핵심구간]  
[핵심 구간]  
1.  $60 \times 6$   
  
[풀이 순서]  
1.  $6 \times 6$  계산  
2. 결과에 10 곱하기  
3. 최종 값 작성  
-----  
[문제] 다음에서 1에서 9까지의 수 중  $\square$  안에 들어갈 수 있는 수의 개수를 구하세요.  $[ 48 \times ]$   
[핵심구간]  
[핵심 구간]  
1.  $48 \times 2$   
2.  $58 \times \square$   
3.  $>$   
4. 1에서 9까지의 수  
  
[풀이 순서]  
1.  $48 \times 2$  계산  
2. 결과값 96과  $58 \times \square$  비교  
3.  $58 \times \square < 96$  조건 만족하는 수 찾기  
4. 1~9 대입 후 가능 여부 확인  
5. 가능한 수 개수 세기  
-----  
[문제]  $2 \times 3$ 의 값을 구하세요.  
[핵심구간]  
[핵심 구간]  
1.  $2 \times 3$   
2. 값을 구하세요  
  
[풀이 순서]  
1. 2와 3을 곱한다  
2. 곱한 결과를 계산한다
```

[QWQ 샘플 10개 출력]

- 핵심 구간 항목이 원문에 실제로 포함된 표현인지 확인
- 형식 준수 확인
 - [핵심 구간] / [풀이 순서] 형식 확인
 - <THINK> 태그 후처리 후 형식 이탈 없음

핵심 구간 추출 프롬프트 개선 과정

v1. 단순 지시

"핵심 구간을 추출해줘" → 형식 없이 풀이 과정과 섞여서 출력됨

v2. 출력 형식 명시 + 분량 제한

"번호 매겨서 핵심 구간만, 설명 금지, 최대 5개" → 형식은 잡혔지만 <THINK> 추론 과정이 출력에 섞임

v3. Few-shot 방식 적용

문제-핵심구간 예시 1개 제시 → 출력 안정성 크게 개선, 환각 거의 사라짐

핵심 구간 추출 프롬프트 개선 과정

v4. 후처리 + 항목 분리 + 입력 확장

- "핵심 구간"은 원문 표현 그대로 사용
 - "풀이 순서"는 모델이 자율 판단하도록 역할 분리
 - V1(문제만) / V2(문제+보기) 두 버전 비교 실험 구현
- 최종 형식 확정

→ 보기에 중점을 둔 문제에서 타겟팅이 약함

```
// — V2: 문제 + 보기 —
function buildPromptV2(problemData) {
  const questionText = (problemData?.questionLines ?? []).join(" ");
  const choicesText = (problemData?.choices ?? []).map((c, i) => {
    const raw = typeof c === "string" ? c : c?.rawText ?? "";
    return `${i+1}. ${raw}`;
  }).join("\n");

  return `당신은 초등학교 수학 교사입니다. 학생이 이 문제를 올바르게 풀기 위해 반드시 시선이 머물러야 할 핵심 부분을 추출합니다.`;
}
```

절차:

1. 머릿속으로 문제를 풀이하세요 (출력하지 말 것)
2. 각 오답 선지가 왜 틀렸는지 추론하세요 (출력하지 말 것)
3. 풀이 과정과 오답 추론을 바탕으로, 반드시 확인해야 할 핵심 구간과 놓치면 오답으로 이어지는 주의 구간을 추출하세요

규칙:

1. 정답이라는 이유만으로 보기를 핵심 구간에 포함하지 말 것
2. 오답이 나오는 원인이 되는 문제 속 표현을 주의 구간에 포함할 것
3. 문제에 있는 표현 그대로만 사용
4. 번호 매겨서 간결하게 출력
5. 각 항목 최대 5개 이내

예시:

문제: 철수는 사탕 15개를 가지고 있었습니다. 친구에게 7개를 주었습니다. 남은 사탕은 몇 개입니까?

보기:

1. 6개
2. 7개
3. 8개
4. 9개

[핵심 구간]

1. 사탕 15개
2. 7개를 주었습니다
3. 남은 사탕은 몇 개

[주의 구간]

1. 남은 - '준 것'이 아니라 '남은 것'을 구해야 함
2. 주었습니다 - 더하기가 아닌 빼기 연산

[풀이 순서]

1. 처음 가진 수 확인
2. 준 수 빼기
3. 답 구하기

문제: \${questionText}

보기:

\${choicesText}`;

}

한 학기 동안 마주한 인프라 문제와 해결

1) GPU 메모리 부족

GPU 6, 7번 모두 타 프로세스 상시 점유

→ 4-bit 양자화 + 외부 추론 서버(API) 활용으로 우회

2) 네트워크/SSH 연결 문제

서버 IP 불일치, 포트 미지정으로 접속 반복 실패

→ SSH config 점검 후 포트를 명시(8510)해 해결

3) CORS 문제

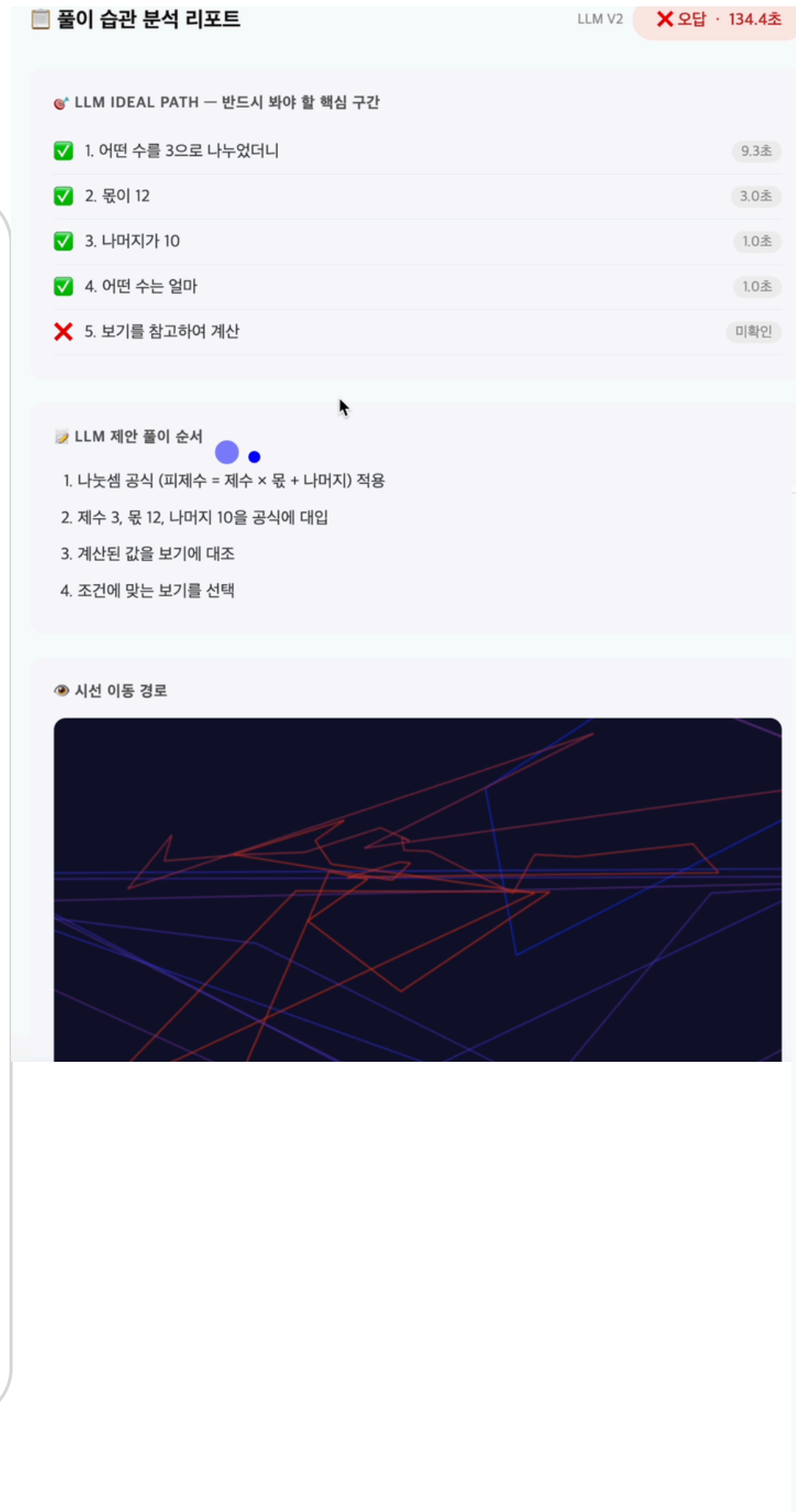
맥 브라우저에서 CyberSecurity-2G서버 직접 호출 불가

→ 8gpu 서버에 프록시 서버(proxy.py) 구축으로 해결

→ 실제 서버 환경에서 발생하는 엔지니어링 문제를 직접 해결한 경험

구현 완료 기능

- AI Hub 초등 수학 데이터셋 4756개 중
텍스트+LaTeX 문제 1032개 필터링 → 50개 실험
- 문제 텍스트 + 보기 자동 파싱 및 화면 표시
- 마우스 호버 기반 시선 대체 → AOI 영역별 체류 시간 기록
- QwQ-32B API 실시간 호출 → 핵심 구간 + 풀이 순서 반환
- 통합 리포트 출력:
 - 마우스 이동 경로 Canvas 시각화
 - 영역별 시선 비율 도넛차트
 - 시선 이동 경로 SVG 시계열
 - 핵심 구간 확인 여부 /
 - LLM 제안 풀이 순서
 - 5가지 오답 패턴 감지
 - AI 처방



시선추적 모듈 연동

[이전] — 마우스 호버 기반 시선 대체

[현재] — 실제 시선추적 장치로 입력 소스 교체

전체 시스템(LLM · AOI 매핑 · 리포트 생성)은
이미 구현 완료된 상태

→ "마우스 좌표"를 "실제 시선 좌표"로 교체하는 입력 소스 전환

[연동 구조]

[시선추적 모듈] 실시간 시선 좌표 / 체류시간 수집



[좌표 → AOI 매핑] 문제·보기 영역과 시선 좌표 대조



[LLM] QwQ-32B가 판단한 핵심 구간과
실제 시선이 머문 AOI를 비교



[리포트] 실제 시선 기반 패턴 진단 리포트 생성



patternDetector.js 5대 패턴 탐지 결과와 통합



→ 핵심 구간 비교 로직은 그대로 유지하면서
입력 소스만 실제 시선추적 장치로 전환

— AOI 시스템 연동 구조

QwQ LLM Ideal Path ↔ AOI 시스템 연동 구조

학생이 답 제출



QwQ-32B → 핵심 구간 + 풀이 순서 생성



시선데이터와 비교

"학생이 봐야 했던 구간" vs "실제로 본 구간"



핵심 구간 확인 여부 판정



patternDetector.js 5대 패턴 탐지 결과와 통합



최종 리포트 출력

시선 추적 + LLM 통합 리포트

[정답 여부 / 총 소요시간]

[핵심 구간 확인 여부 (시선 체류 기준)]

-  핵심 구간 1 (응시 확인)
-  핵심 구간 2 (응시 없음 → 패턴 후보)

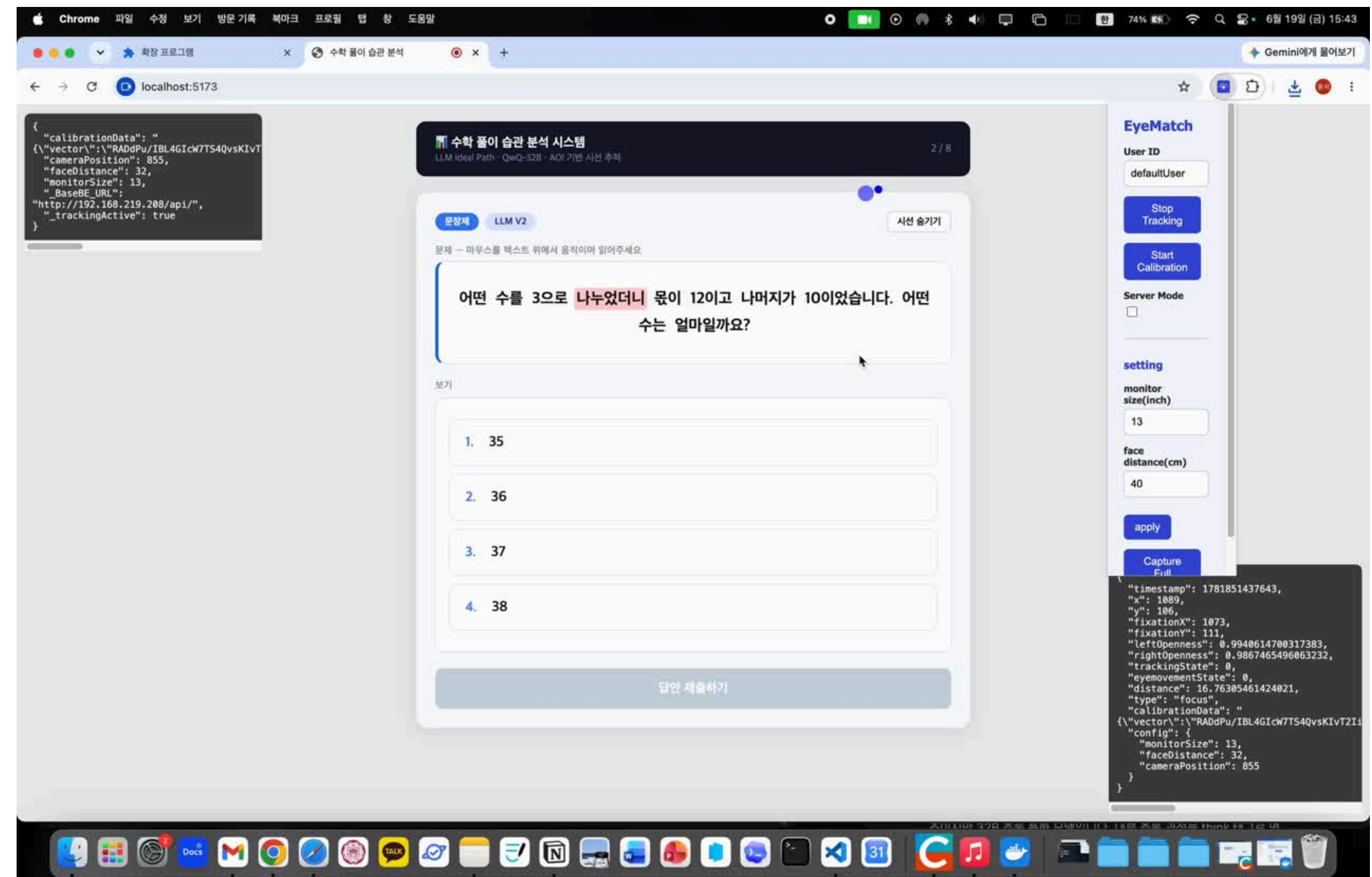
[시선 경로 시각화]

- 영역별 응시 시간 도넛차트
- 시간 흐름에 따른 시선 이동 SVG 경로

[AI 처방]

- 미응시 구간을 근거로 한 맞춤 AI 피드백 생성

→ 구조와 로직은 마우스 데모에서 이미 검증 완료
연동 후 실제 시선 데이터로 동일하게 동작



한 학기 전체 성취

- ✓ 5대 오답 패턴 정의 및 진단 알고리즘 설계 (1차)
- ✓ AOI 기반 아키텍처 분리 구현 (2차)
 - useGazeTracker / focusLog / patternDetector
- ✓ AIHub 데이터 변환 파이프라인 구축 (3차)
- ✓ LLM 모델 선정 실패 경험 → 원인 분석 → 대체 모델 전환 (4차)
- ✓ 4단계 프롬프트 엔지니어링으로 출력 안정화 (4차)
- ✓ 인프라 이슈 3가지 해결 (4차)
- ✓ QwQ-32B LLM Ideal Path ↔ AOI 시스템 실제 코드 통합 (4차)
- ✓ V1/V2 프롬프트 버전 비교 실험 구현 (4차)
- ✓ 실제 시선 추적 연동 - Chrome 확장 프로그램 (4차)

한계점 및 향후 계획

1. AOI 크기 및 인식 범위 조정

현재 | AOI 영역 협소로 인한 미세 시선 떨림 흡수 불가 및 인식 실패

향후 | AOI 인식 범위 확장(마진 조정) 및 입력 감도 조율을 통한 인식률 향상

2. 데이터 규모

현재 | 50개 추출 후 실험 (텍스트 + Latex 문제 1,032개 중)

향후 | 전체 4,075개로 확장 (이미지+Latex 문제)

3. 패턴 최종 판정

현재 | LLM 1회 판정

향후 | LLM as a Judge 방식 도입 (QwQ 출력 결과도 별도 LLM이 재검증하여 판정 신뢰도 향상)

한계점 및 향후 계획

4. 이미지 문제

현재 | 텍스트/LaTex만 처리

향후 | ViT/DINOv2 기반 확장

-
- **DINOv2는 이미지 내 여러 부분의 관계를 학습하는 방식**
 - attention map으로 핵심 영역이 자연스럽게 드러남
 - I-JEPA 대비 핵심 영역 탐지에 더 적합할 것으로 판단
 - ⇒ **DINOv2 기반 우선 실험 예정**

감사합니다.